

Vectors, matrices, tensors, NumPy, and PyTorch

COMP364, Dickinson College

John MacCormick

August 2, 2026

1 Matrices

A *matrix* is a two-dimensional array of numbers, such as

$$M = \begin{pmatrix} 5 & 0 & -7.3 & 0 \\ 2 & 1.1 & -5.66 & -1 \\ 6 & 9.1 & 9.1 & -1 \end{pmatrix}.$$

In this course, we use two common Python packages for manipulating vectors and matrices: NumPy and PyTorch. To create the matrix M above in NumPy, we can use the following syntax.

```
1 import numpy as np
2
3 M_numpy = np.array([
4     [5, 0, -7.3, 0],
5     [2, 1.1, -5.66, -1],
6     [6, 9.1, 9.1, -1],
7 ])
```

To create the same matrix in PyTorch, we can convert the NumPy matrix

```
1 import torch
2
3 M_torch = torch.from_numpy(M_numpy)
```

or create it directly as follows.

```
1 M_torch = torch.tensor([
2     [5, 0, -7.3, 0],
3     [2, 1.1, -5.66, -1],
4     [6, 9.1, 9.1, -1],
5 ], dtype=torch.float64)
```

A *tensor* is the term used by PyTorch for a multidimensional array. We will study the tensor data type later.

Shape of a matrix

The matrix M above has three rows and four columns. We say it is a “three-by-four” matrix, written 3×4 . The row count is always given first, so a matrix with r rows and c columns has *shape* $r \times c$. Sometimes we will write the matrix size directly under the matrix name, as in $M_{3 \times 4}$.

In NumPy, we can print the shape of M as follows.

```
1 print(M_numpy.shape)
```

This prints

```
1 (3, 4)
```

In PyTorch, the corresponding code is

```
1 print(M_torch.shape)
```

which prints

```
1 torch.Size([3, 4])
```

Matrix elements and indices

In this document and in this course, we number our rows and columns beginning from 0, not 1. So the matrix M shown above has row 0 equal to $(5, 0, -7.3, 0)$, row 2 equal to $(6, 9.1, 9.1, -1)$, and column 1 equal to

$$\begin{pmatrix} 0 \\ 1.1 \\ 9.1 \end{pmatrix}.$$

The indices of individual elements also begin from 0, not 1. Sometimes we denote an element by writing its row and column indices as subscripts directly on the matrix label. For example,

$$M_{02} = -7.3.$$

Alternatively, we can use a different symbol for the elements of a matrix. We might write

$$M = (m_{ij}),$$

where m_{ij} denotes the element in row i and column j . For example, $m_{13} = -1$.

Column vectors, row vectors, and 1D-shaped vectors

A column vector is the same as a matrix with one column, and a row vector is the same as a matrix with one row. For example,

$$v_{3 \times 1} = \begin{pmatrix} 2 \\ -1 \\ 4 \end{pmatrix} \quad \text{and} \quad w_{1 \times 3} = (2 \quad -1 \quad 4)$$

are a column vector and a row vector, respectively. The dimensions beneath each vector explicitly give its number of rows and columns.

In NumPy, we can create the column vector v and the row vector w as follows.

```
1 v_numpy = np.array([
2     [ 2],
3     [-1],
4     [ 4],
5 ])
6 w_numpy = np.array([[2, -1, 4]])
```

Similarly, in PyTorch we can write

```
1 v_torch = torch.tensor([
2     [ 2],
3     [-1],
4     [ 4],
5 ])
6 w_torch = torch.tensor([[2, -1, 4]])
```

We can also extract a column vector or row vector from the matrix M . For example, the following NumPy code extracts column 1 and row 2:

```
1 column_1_numpy = M_numpy[:, 1:2]
2 row_2_numpy = M_numpy[2:3, :]
```

The corresponding PyTorch code is

```
1 column_1_torch = M_torch[:, 1:2]
2 row_2_torch = M_torch[2:3, :]
```

Using slices such as `1:2` and `2:3`, rather than single indices, preserves the two-dimensional shapes of the column and row vectors.

Sometimes we employ a vector that is not specifically oriented as a row or a column. We call this a *1D-shaped* vector. If we need to emphasize that a particular vector is 1D-shaped, we will write its shape as $n \times$, where n is the number of elements in the vector. For example, $\underset{3 \times}{u} = (2, -1, 4)$ is a 1D-shaped vector with three elements.

In NumPy, a single pair of brackets creates a 1D-shaped vector:

```
1 u_numpy = np.array([2, -1, 4])
2 print(u_numpy.shape)
```

This prints `(3,)`, NumPy's notation for a 1D-shaped array with three elements. In PyTorch, we can write

```
1 u_torch = torch.tensor([2, -1, 4])
2 print(u_torch.shape)
```

which prints `torch.Size([3])`.

Using a single integer index to select a row or column of M produces a 1D-shaped vector. In NumPy, for example, we can write

```

1 column_1_numpy_1d = M_numpy[:, 1]
2 row_2_numpy_1d = M_numpy[2, :]
3
4 print(column_1_numpy_1d)      # [0.  1.1  9.1]
5 print(column_1_numpy_1d.shape) # (3,)
6 print(row_2_numpy_1d)        # [ 6.   9.1  9.1 -1. ]
7 print(row_2_numpy_1d.shape)   # (4,)
```

The same indexing syntax works in PyTorch:

```

1 column_1_torch_1d = M_torch[:, 1]
2 row_2_torch_1d = M_torch[2, :]
3
4 print(column_1_torch_1d.shape) # torch.Size([3])
5 print(row_2_torch_1d.shape)   # torch.Size([4])
```

Unlike the slices used above to preserve rows and columns as two-dimensional matrices, integer indices remove the selected dimension.

Note that the size or shape of a matrix is itself represented by a 1D-shaped sequence in NumPy and has its own data type in PyTorch, which resembles a 1D-shaped vector. For example, in NumPy we have

```

1 M_numpy_shape = M_numpy.shape
2
3 print(M_numpy_shape)      # (3, 4)
4 print(type(M_numpy_shape)) # <class 'tuple'>
5 print(len(M_numpy_shape)) # 2
```

Thus, the shape is a 1D-shaped tuple containing the number of rows and the number of columns. In PyTorch, the corresponding example is

```

1 M_torch_size = M_torch.size()
2
3 print(M_torch_size)      # torch.Size([3, 4])
4 print(type(M_torch_size)) # <class 'torch.Size'>
5 print(len(M_torch_size)) # 2
```

The `torch.Size` object is a tuple-like, 1D-shaped sequence whose elements give the size of each dimension of the tensor.

Notation for matrix rows and columns

When referring to specific rows or columns of a matrix, we use the following Python-based notation:

- $M_{i,:}$ is a row vector that denotes the i -th row of matrix M .
- $M_{:,j}$ is a column vector that denotes the j -th column of matrix M .

As an example, take M to be the same 3×4 matrix as earlier:

$$M = \begin{pmatrix} 5 & 0 & -7.3 & 0 \\ 2 & 1.1 & -5.66 & -1 \\ 6 & 9.1 & 9.1 & -1 \end{pmatrix}.$$

3×4

Then row 2 of M is the 1×4 row vector

$$M_{2,:} = (6 \quad 9.1 \quad 9.1 \quad -1).$$

1×4

while column 1 is the 3×1 column vector

$$M_{:,1} = \begin{pmatrix} 0 \\ 1.1 \\ 9.1 \end{pmatrix}.$$

3×1

We will sometimes abuse this row-and-column notation by allowing the same notation to represent a 1D-shaped vector instead of a column or row vector. For example, $M_{2,:}$ could have shape $4 \times$ rather than 1×4 , and $M_{:,1}$ could have shape $3 \times$ rather than 3×1 . The context will usually make it clear which shape is intended. Sometimes we may explicitly add the shape below the vector to avoid ambiguity, as in $M_{2,:}$ or $M_{2,:}$.

$4 \times$ 1×4

0D-, 1D-, and 2D-shaped arrays and tensors

As we already know, a matrix is a two-dimensional array of numbers. Matrices are a particular case of a more general data structure that can store an array of numbers of any dimension, sometimes called a *multidimensional array* or a *tensor*. Numpy uses the technical term *array* (Python datatype `numpy.ndarray`) for this data structure, while PyTorch uses the technical term *tensor* (Python datatype `torch.Tensor`). In this document, we will use the terms *array* and *tensor* interchangeably, although we will generally prefer *tensor*.

Tensors can have dimension zero, one, two, three, and so on. We will study tensors of dimension three or more later. For now, let's restrict attention to tensors of dimension zero, one, and two. Tensors of dimension two are matrices. Tensors of dimension one are what we have been calling 1D-shaped vectors.

What about a tensor of dimension zero? This is called a *scalar*. In NumPy and PyTorch, a scalar tensor is not exactly the same thing as a raw number. To extract the number stored inside a scalar tensor for use as a regular Python numeric variable, we use the `item` method. In NumPy:

```
1 x_numpy = np.array(3.14)
2
3 print(x_numpy.shape)  # ()
```

```

4 x = x_numpy.item()
5 print(x)           # 3.14
6 print(type(x))     # <class 'float'>

```

In PyTorch, we can do the same thing:

```

1 x_torch = torch.tensor(3.14, dtype=torch.float64)
2
3 print(x_torch.shape) # torch.Size([])
4 x = x_torch.item()
5 print(x)           # 3.14
6 print(type(x))     # <class 'float'>

```

PyTorch and NumPy offer many ways of converting arrays or tensors from one dimensionality to another. Here are a few examples, beginning with NumPy:

```

1 # Convert a 0D array to a 1D-shaped array.
2 scalar_numpy = np.array(7)
3 one_d_numpy = np.atleast_1d(scalar_numpy)
4 print(scalar_numpy.shape) # ()
5 print(one_d_numpy.shape)  # (1,)
6
7 # Convert a 1D-shaped vector to a row or column vector.
8 u_numpy = np.array([1, 2, 3])
9 row_numpy = u_numpy.reshape(1, -1) # -1 means "infer this dimension"
10 column_numpy = u_numpy.reshape(-1, 1) # -1 means "infer this dimension"
11 print(row_numpy.shape)    # (1, 3)
12 print(column_numpy.shape) # (3, 1)
13
14 # Remove a dimension of size 1.
15 u_again_numpy = column_numpy.squeeze()
16 print(u_again_numpy.shape) # (3,)
17
18 # Flatten a matrix into a 1D-shaped vector.
19 A_numpy = np.array([[1, 2], [3, 4]])
20 flat_numpy = A_numpy.flatten()
21 print(flat_numpy)         # [1 2 3 4]
22 print(flat_numpy.shape)   # (4,)

```

PyTorch provides similar operations:

```

1 # Convert a 0D tensor to a 1D-shaped tensor.
2 scalar_torch = torch.tensor(7)
3 one_d_torch = scalar_torch.unsqueeze(0)
4 print(scalar_torch.shape) # torch.Size([])
5 print(one_d_torch.shape)  # torch.Size([1])
6
7 # Convert a 1D-shaped vector to a row or column vector.
8 u_torch = torch.tensor([1, 2, 3])

```

```

9 row_torch = u_torch.unsqueeze(0)
10 column_torch = u_torch.unsqueeze(1)
11 print(row_torch.shape)      # torch.Size([1, 3])
12 print(column_torch.shape)   # torch.Size([3, 1])
13
14 # Remove a dimension of size 1.
15 u_again_torch = column_torch.squeeze(1)
16 print(u_again_torch.shape) # torch.Size([3])
17
18 # Flatten a matrix into a 1D-shaped vector.
19 A_torch = torch.tensor([[1, 2], [3, 4]])
20 flat_torch = A_torch.flatten()
21 print(flat_torch)           # tensor([1, 2, 3, 4])
22 print(flat_torch.shape)     # torch.Size([4])

```

2 Matrix and vector addition

Matrix and vector addition is performed element-wise. For two matrices A and B of the same dimensions, the sum $C = A + B$ is computed as:

$$c_{ij} = a_{ij} + b_{ij}$$

for all i and j . Similarly, for 1D-shaped vectors u and v of the same length, the sum $w = u + v$ is computed as:

$$w_i = u_i + v_i$$

for all i .

In NumPy, the $+$ operator performs this element-wise addition:

```

1 A_numpy = np.array([
2     [1, 2],
3     [3, 4],
4 ])
5 B_numpy = np.array([
6     [5, 6],
7     [7, 8],
8 ])
9 C_numpy = A_numpy + B_numpy
10 print(C_numpy)
11 # [[ 6  8]
12 #   [10 12]]
13
14 u_numpy = np.array([1, 2, 3])
15 v_numpy = np.array([4, 5, 6])
16 w_numpy = u_numpy + v_numpy
17 print(w_numpy) # [5 7 9]

```

PyTorch uses the same operator and produces the same element-wise results:

```

1 A_torch = torch.tensor([
2     [1, 2],
3     [3, 4],
4 ])
5 B_torch = torch.tensor([
6     [5, 6],
7     [7, 8],
8 ])
9 C_torch = A_torch + B_torch
10 print(C_torch)
11 # tensor([[ 6,  8],
12 #         [10, 12]])
13
14 u_torch = torch.tensor([1, 2, 3])
15 v_torch = torch.tensor([4, 5, 6])
16 w_torch = u_torch + v_torch
17 print(w_torch) # tensor([5, 7, 9])

```

3 Broadcasting

There is a natural and obvious way of adding a constant to a matrix or vector: simply add the constant to each element. For example, if we have a matrix M and we want to add a constant a to it, we compute $M' = M + a$ by setting

$$m'_{ij} = m_{ij} + a$$

for all i and j . One way to think about this is that the constant a gets expanded into a matrix of the same shape as M , and then the two matrices are added element-wise as in this example:

$$\begin{aligned}
 \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} + 10 &= \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} + \begin{pmatrix} 10 & 10 \\ 10 & 10 \end{pmatrix} \\
 &= \begin{pmatrix} 11 & 12 \\ 13 & 14 \end{pmatrix}.
 \end{aligned}$$

The technical term for this kind of expansion is *broadcasting*. We say the value a has been *broadcast* to the shape of the matrix M .

However, broadcasting works with more complex situations also. For example, a column vector can be broadcast to a matrix with the same number of rows:

$$\begin{aligned}
 \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} + \begin{pmatrix} 10 \\ 20 \end{pmatrix} &= \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} + \begin{pmatrix} 10 & 10 & 10 \\ 20 & 20 & 20 \end{pmatrix} \\
 &= \begin{pmatrix} 11 & 12 & 13 \\ 24 & 25 & 26 \end{pmatrix}.
 \end{aligned}$$

And a row vector can be broadcast to a matrix with the same number of columns:

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} + (10 \ 20 \ 30) = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} + \begin{pmatrix} 10 & 20 & 30 \\ 10 & 20 & 30 \end{pmatrix} \\ = \begin{pmatrix} 11 & 22 & 33 \\ 14 & 25 & 36 \end{pmatrix}.$$

NumPy and PyTorch support broadcasting. For example, NumPy automatically broadcasts a column vector in the following addition:

```
1 A_numpy = np.array([[1, 2, 3],
2                     [4, 5, 6]])
3 b_numpy = np.array([[10],
4                     [20]])
5 print(A_numpy + b_numpy)
6 # [[11 12 13]
7 #   [24 25 26]]
```

PyTorch applies the same broadcasting rule:

```
1 A_torch = torch.tensor([[1, 2, 3],
2                          [4, 5, 6]])
3 b_torch = torch.tensor([[10],
4                          [20]])
5 print(A_torch + b_torch)
6 # tensor([[11, 12, 13],
7 #         [24, 25, 26]])
```

Broadcasting can be rather complex. For example, make sure you understand what happens when a column vector is added to a row vector, as in this example:

$$\begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}_{3 \times 1} + (10 \ 20 \ 30 \ 40)_{1 \times 4} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 2 & 2 & 2 & 2 \\ 3 & 3 & 3 & 3 \end{pmatrix} + \begin{pmatrix} 10 & 20 & 30 & 40 \\ 10 & 20 & 30 & 40 \\ 10 & 20 & 30 & 40 \end{pmatrix} \\ = \begin{pmatrix} 11 & 21 & 31 & 41 \\ 12 & 22 & 32 & 42 \\ 13 & 23 & 33 & 43 \end{pmatrix}_{3 \times 4}.$$

The column vector is broadcast across four columns, while the row vector is broadcast down three rows. Consequently, adding shapes 3×1 and 1×4 produces a matrix of shape 3×4 .

NumPy and PyTorch support more complex forms of broadcasting, in addition to the simple cases studied here. It's an extremely powerful tool, but it can also be confusing. For more information, see the NumPy documentation on broadcasting: <https://numpy.org/doc/stable/user/basics.broadcasting.html>.

4 Vector dot product

The dot product

We introduce the vector dot product via a simple application. Suppose we sell three resources in a store: wheat, sheep, and ore. Wheat costs \$3 per unit, sheep costs \$8 per unit, and ore costs \$4 per unit. We can represent the prices of these resources as the 1D-shaped price vector $p = (3, 8, 4)$.
 Now suppose customer Sofia wants 2 units of wheat, 1 unit of sheep, and 3 units of ore. We can represent this order as the 1D-shaped order vector $s = (2, 1, 3)$. The total cost for Sofia is

$$3 \times 2 + 8 \times 1 + 4 \times 3 = 6 + 8 + 12 = 26.$$

This calculation is defined to be the *dot product* of p and s , written $p \cdot s$. In general, the dot product of two 1D-shaped vectors with the same number of elements is computed by multiplying corresponding elements and summing the results. For example, if $u = (u_0, u_1, \dots, u_{n-1})$ and $v = (v_0, v_1, \dots, v_{n-1})$, then their dot product is

$$u \cdot v = u_0v_0 + u_1v_1 + \dots + u_{n-1}v_{n-1} = \sum_{i=0}^{n-1} u_i v_i.$$

Dot products have many applications beyond the “price dot order” example above. For example, if p is the vector of probabilities for the outcomes in a gambling game (so p ’s elements sum to 1) and r is the vector of rewards for those outcomes, then the expected reward is given by the dot product $p \cdot r$. Another example arises in neural networks, where the output of a neuron is often computed as the weighted sum of its inputs. If the weights are expressed as a vector w , and the inputs are a vector x , then the output is often computed as $w \cdot x + b$, where b is a *bias* term.

In NumPy, we can compute the dot product with `np.dot` or the `@` operator:

```
1 p_numpy = np.array([3, 8, 4])
2 s_numpy = np.array([2, 1, 3])
3
4 print(np.dot(p_numpy, s_numpy)) # 26
5 print(p_numpy @ s_numpy)       # 26
```

PyTorch provides the analogous `torch.dot` function and also supports the `@` operator:

```
1 p_torch = torch.tensor([3, 8, 4])
2 s_torch = torch.tensor([2, 1, 3])
3
4 print(torch.dot(p_torch, s_torch)) # tensor(26)
5 print(p_torch @ s_torch)          # tensor(26)
```

In both cases, the result is a scalar rather than a vector.

Vector length and norm

By the Pythagorean theorem, the length of a 1D-shaped, two-element vector v is $\sqrt{v_0^2 + v_1^2}$. For a three-element vector, it is $\sqrt{v_0^2 + v_1^2 + v_2^2}$. Mathematicians generalize this notion of length to n dimensions, so that a 1D-shaped vector with n elements is defined to have length equal to the square root of the sum of its squared elements. Importantly, this can also be expressed using the dot product of a vector with itself. Our notation for the length of a vector is $\|v\|$, and we have

$$\|v\| = \sqrt{\sum_{i=0}^{n-1} v_i^2} = \sqrt{v \cdot v}.$$

For example, consider the vector $v = (3, 4)$. Its length is

$$\|v\| = \sqrt{3^2 + 4^2} = \sqrt{9 + 16} = \sqrt{25} = 5.$$

In NumPy, we can compute this length either directly from the dot product or with `np.linalg.norm`:

```
1 v_numpy = np.array([3, 4])
2
3 print(np.sqrt(v_numpy @ v_numpy)) # 5.0
4 print(np.linalg.norm(v_numpy))   # 5.0
```

PyTorch provides the corresponding operations:

```
1 v_torch = torch.tensor([3.0, 4.0])
2
3 print(torch.sqrt(v_torch @ v_torch)) # tensor(5.)
4 print(torch.linalg.vector_norm(v_torch)) # tensor(5.)
```

As you can see, the word *norm* can be synonymous with *length* in this context.

Unit vectors and normalization

If we divide a vector by its length, we obtain a vector whose length is one. This is called a *unit vector*, and we often say that we have *normalized* the vector.

For example, the vector $v = (3, 4)$ has length 5, so its normalized version is

$$\hat{v} = \frac{v}{\|v\|} = \frac{(3, 4)}{5} = \left(\frac{3}{5}, \frac{4}{5}\right).$$

Indeed,

$$\|\hat{v}\| = \sqrt{\left(\frac{3}{5}\right)^2 + \left(\frac{4}{5}\right)^2} = 1.$$

In NumPy, we can normalize the vector as follows:

```

1 unit_v_numpy = v_numpy / np.linalg.norm(v_numpy)
2
3 print(unit_v_numpy)           # [0.6 0.8]
4 print(np.linalg.norm(unit_v_numpy)) # 1.0

```

The corresponding PyTorch code is

```

1 unit_v_torch = v_torch / torch.linalg.vector_norm(v_torch)
2
3 print(unit_v_torch)           # tensor([0.6000, 0.8000])
4 print(torch.linalg.vector_norm(unit_v_torch)) # tensor(1.)

```

Angles between vectors

It is an extremely important fact that the dot product of two unit vectors is equal to the cosine of the angle between them. That is, if $\hat{u}$ and $\hat{v}$ are unit vectors and θ is the angle between them, then

$$\hat{u} \cdot \hat{v} = \cos \theta.$$

For vectors that are not necessarily of unit length, the same equation holds, but we must multiply by their lengths. Thus, for arbitrary nonzero vectors u and v , we have

$$u \cdot v = \|u\| \|v\| \cos \theta.$$

This holds for 1D-shaped vectors of any length. Although it is difficult to visualize angles between vectors in a high-dimensional space, the formula above can serve as the definition of the angle between vectors. This is often used in machine learning to measure the similarity between vectors. It is an idea known as *cosine similarity*.

Cosine similarity

Solving the preceding equation for $\cos \theta$ gives the cosine similarity of two nonzero vectors:

$$\text{cosim}(u, v) = \cos \theta = \frac{u \cdot v}{\|u\| \|v\|}.$$

Its value ranges from -1 to 1 . Values near 1 indicate vectors pointing in similar directions, values near 0 indicate nearly perpendicular directions, and values near -1 indicate opposing directions.

First, consider some examples in two dimensions. Let $u = (1, 0)$. The vector $v = (2, 0)$ points in exactly the same direction as u , so

$$\text{cosim}(u, v) = \frac{(1)(2) + (0)(0)}{(1)(2)} = 1.$$

The vectors have different lengths, but cosine similarity ignores this difference and measures only their directions. The vector $w = (1, 1)$ points in a somewhat different direction:

$$\text{cosim}(u, w) = \frac{(1)(1) + (0)(1)}{(1)(\sqrt{2})} = \frac{1}{\sqrt{2}} \approx 0.707.$$

The vector $x = (0, 1)$ is perpendicular to u , giving

$$\text{cosim}(u, x) = 0,$$

while $y = (-1, 0)$ points in exactly the opposite direction, giving

$$\text{cosim}(u, y) = -1.$$

The same ideas apply in four dimensions, even though we cannot visualize the vectors as easily. Let $a = (1, 1, 0, 0)$. The vector $b = (2, 2, 0, 0)$ points in the same direction, so

$$\text{cosim}(a, b) = 1.$$

For $c = (1, 0, 1, 0)$, we obtain an intermediate similarity:

$$\text{cosim}(a, c) = \frac{1}{(\sqrt{2})(\sqrt{2})} = \frac{1}{2}.$$

The vector $d = (0, 0, 1, 1)$ has no nonzero components in common with a , so $a \cdot d = 0$ and

$$\text{cosim}(a, d) = 0.$$

Finally, $e = (-1, -1, 0, 0)$ points in exactly the opposite direction from a , and therefore

$$\text{cosim}(a, e) = -1.$$

For example, we can compute the cosine similarity between $a = (1, 1, 0, 0)$ and $c = (1, 0, 1, 0)$ in NumPy as follows:

```

1 a_numpy = np.array([1, 1, 0, 0])
2 c_numpy = np.array([1, 0, 1, 0])
3
4 similarity = (a_numpy @ c_numpy) / (
5     np.linalg.norm(a_numpy) * np.linalg.norm(c_numpy)
6 )
7 print(similarity)  # 0.4999999999999999
```

The corresponding PyTorch calculation is

```

1 a_torch = torch.tensor([1.0, 1.0, 0.0, 0.0])
2 c_torch = torch.tensor([1.0, 0.0, 1.0, 0.0])
3
4 similarity = torch.nn.functional.cosine_similarity(
5     a_torch, c_torch, dim=0
6 )
7 print(similarity)  # tensor(0.5000)
```

5 Matrix multiplication

Matrix multiplication can seem weird if you've never seen it before. Here we introduce it via a simple application. Let's return to the earlier example in which we are selling three resources in a store: wheat, sheep, and ore. Wheat costs \$3 per unit, sheep costs \$8 per unit, and ore costs \$4 per unit. Previously, in the section on dot products, we represented these prices as a *1D-shaped* vector $p = (3, 8, 4)$. Now, we instead represent the prices of these resources as a *column* vector:

$$p = \begin{pmatrix} 3 \\ 8 \\ 4 \end{pmatrix}.$$

In the earlier example there was only a single customer, but now we imagine there are two customers. As before, customer Sofia wants 2 units of wheat, 1 unit of sheep, and 3 units of ore. Now we add customer Thomas, who wants 1 unit of wheat, 4 units of sheep, and 2 units of ore. We can represent these orders as a matrix with Sofia's order in row 0 and Thomas's order in row 1:

$$O = \begin{pmatrix} 2 & 1 & 3 \\ 1 & 4 & 2 \end{pmatrix}.$$

The total cost for Sofia is $2 \cdot 3 + 1 \cdot 8 + 3 \cdot 4 = 6 + 8 + 12 = 26$, while the total cost for Thomas is $1 \cdot 3 + 4 \cdot 8 + 2 \cdot 4 = 3 + 32 + 8 = 43$. This motivates us to define the matrix product of O and p as a new column vector c whose elements are the total costs for each customer:

$$c = Op = \begin{pmatrix} 2 & 1 & 3 \\ 1 & 4 & 2 \end{pmatrix} \begin{pmatrix} 3 \\ 8 \\ 4 \end{pmatrix} = \begin{pmatrix} 2 \cdot 3 + 1 \cdot 8 + 3 \cdot 4 \\ 1 \cdot 3 + 4 \cdot 8 + 2 \cdot 4 \end{pmatrix} = \begin{pmatrix} 26 \\ 43 \end{pmatrix}.$$

Recall our notation $A_{i,j}$ for the i th row of a matrix A . Using this notation, we can express the i th element of the product $c = Op$ as $O_{i,:} \cdot p$ —the dot product of O 's i th row with p . This motivates the general definition multiplying a matrix A by a column vector v :

$$A \times v = \begin{pmatrix} A_{0,:} \cdot v \\ A_{1,:} \cdot v \\ \vdots \\ A_{m-1,:} \cdot v \end{pmatrix}.$$

The simple rule is: every entry in a matrix multiplication is the dot product of a row vector and a column vector.

Let's generalize the application to see how we multiply two matrices rather than a matrix and a vector. For this, imagine we have two sets of prices for the three resources: the previous set of prices for pickup from the store, and a new set of prices for that include delivery. We can represent these prices as a 3×2 matrix:

$$P = \begin{pmatrix} 3 & 5 \\ 8 & 10 \\ 4 & 6 \end{pmatrix}$$

where column 0 is the pickup prices and column 1 is the delivery prices. Now, if we multiply the 2×3 order matrix O by this 3×2 price matrix P , we get a 2×2 matrix whose first column is the total cost for each customer for pickup, and whose second column is the total cost for each customer for delivery:

$$OP = \begin{pmatrix} 2 & 1 & 3 \\ 1 & 4 & 2 \end{pmatrix} \begin{pmatrix} 3 & 5 \\ 8 & 10 \\ 4 & 6 \end{pmatrix} = \begin{pmatrix} 26 & 38 \\ 43 & 66 \end{pmatrix}.$$

The simple rule stated earlier still holds: every entry in result matrix is the dot product of a row vector and a column vector. For example, the entry in row 0, column 1 of the result is $O_{0,:} \cdot P_{:,1} = 2 \cdot 5 + 1 \cdot 10 + 3 \cdot 6 = 38$.

This motivates the general definition of multiplying a matrix A by a matrix B to obtain a matrix C :

$$C_{m \times p} = A_{m \times n} \times B_{n \times p}, \quad \text{where} \quad c_{ij} = A_{i,:} \cdot B_{:,j}.$$

This is more commonly written as $c_{ij} = \sum_{k=0}^{n-1} a_{ik}b_{kj}$, but the equivalent product definition given above is conceptually more useful: c_{ij} is the dot product of the i th row of A and the j th column of B .

Note that in the matrix product $C = AB$, the inner dimensions of A and B must match. That is, if A has n columns, then B must have n rows. We can think of this inner dimension n as being eliminated during the product, leaving only the outer dimensions, which give the shape of the resulting matrix. Thus, if A has shape $m \times n$ and B has shape $n \times p$, then the inner dimension n is eliminated and C has shape $m \times p$:

$$(m \times n)(n \times p) \longrightarrow m \times p.$$

Some examples should make this clear. First, multiplying a 2×3 matrix by a 3×2 matrix produces a 2×2 matrix:

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}_{2 \times 3} \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}_{3 \times 2} = \begin{pmatrix} 4 & 5 \\ 10 & 11 \end{pmatrix}_{2 \times 2}.$$

As another example, multiplying a 3×2 matrix by a 2×4 matrix produces a 3×4 matrix:

$$\begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}_{3 \times 2} \begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \end{pmatrix}_{2 \times 4} = \begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 6 & 8 & 10 & 12 \end{pmatrix}_{3 \times 4}.$$

Column vectors and row vectors use exactly the same definition for matrix multiplication and do not need any special treatment. For example, a row vector can multiply a larger matrix:

$$\begin{pmatrix} 1 & 2 & 3 \end{pmatrix}_{1 \times 3} \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}_{3 \times 2} = \begin{pmatrix} 4 & 5 \end{pmatrix}_{1 \times 2}.$$

Similarly, a larger matrix can multiply a column vector:

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}_{2 \times 3} \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}_{3 \times 1} = \begin{pmatrix} 4 \\ 10 \end{pmatrix}_{2 \times 1}.$$

A row vector multiplied by a column vector produces a 1×1 matrix:

$$\begin{pmatrix} 1 & 2 & 3 \end{pmatrix}_{1 \times 3} \begin{pmatrix} 30 \\ 20 \\ 10 \end{pmatrix}_{3 \times 1} = \begin{pmatrix} 100 \end{pmatrix}_{1 \times 1}.$$

In the opposite order, a column vector multiplied by a row vector produces a larger matrix:

$$\begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}_{3 \times 1} \begin{pmatrix} 4 & 5 \end{pmatrix}_{1 \times 2} = \begin{pmatrix} 4 & 5 \\ 8 & 10 \\ 12 & 15 \end{pmatrix}_{3 \times 2}.$$

In NumPy, the `@` operator performs matrix multiplication. For example,

```
1 A_numpy = np.array([
2     [1, 0],
3     [0, 1],
4     [1, 1],
5 ])
6 B_numpy = np.array([
7     [1, 2, 3, 4],
8     [5, 6, 7, 8],
9 ])
10 C_numpy = A_numpy @ B_numpy
11
12 print(C_numpy)
13 # [[ 1  2  3  4]
14 #   [ 5  6  7  8]
15 #   [ 6  8 10 12]]
16 print(C_numpy.shape) # (3, 4)
```

PyTorch uses the same operator:

```
1 A_torch = torch.tensor([
2     [1, 0],
3     [0, 1],
4     [1, 1],
5 ])
6 B_torch = torch.tensor([
7     [1, 2, 3, 4],
8     [5, 6, 7, 8],
```



```

9  ])
10 C_torch = A_torch @ B_torch
11
12 print(C_torch)
13 # tensor([[ 1,  2,  3,  4],
14 #         [ 5,  6,  7,  8],
15 #         [ 6,  8, 10, 12]])
16 print(C_torch.shape) # torch.Size([3, 4])

```

6 Matrix transposition

The *transpose* of a matrix is obtained by swapping its rows and columns. For a matrix A with dimensions $m \times n$, its transpose $A^\top$ has dimensions $n \times m$. The element in the i -th row and j -th column of A becomes the element in the j -th row and i -th column of $A^\top$.

For example, if we have a matrix

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix},$$

then its transpose is

$$A^\top = \begin{pmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{pmatrix}.$$

Transposition converts a row vector into a column vector:

$$\begin{pmatrix} 1 & 2 & 3 \end{pmatrix}_{1 \times 3}^\top = \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}_{3 \times 1}.$$

Likewise, it converts a column vector into a row vector:

$$\begin{pmatrix} 4 \\ 5 \\ 6 \end{pmatrix}_{3 \times 1}^\top = \begin{pmatrix} 4 & 5 & 6 \end{pmatrix}_{1 \times 3}.$$

In NumPy, the `.T` attribute returns the transpose of a matrix:

```

1  A_numpy = np.array([
2      [1, 2, 3],
3      [4, 5, 6],
4  ])
5  A_transpose_numpy = A_numpy.T
6
7  print(A_transpose_numpy)
8  # [[1 4]

```

```

9 # [2 5]
10 # [3 6]]
11 print(A_transpose_numpy.shape) # (3, 2)

```

PyTorch uses the same attribute for two-dimensional tensors:

```

1 A_torch = torch.tensor([
2     [1, 2, 3],
3     [4, 5, 6],
4 ])
5 A_transpose_torch = A_torch.T
6
7 print(A_transpose_torch)
8 # tensor([[1, 4],
9 #         [2, 5],
10 #         [3, 6]])
11 print(A_transpose_torch.shape) # torch.Size([3, 2])

```

These examples use two-dimensional arrays. Transposing a 1D-shaped vector does not turn it into a row or column vector; a dimension must first be added, for example with `reshape` or `unsqueeze`.

7 Three-Dimensional Tensors

As a simple example of a three-dimensional tensor, let us return to the example of a store selling three resources. We already have a price matrix giving the prices of the three resources in two different situations: pickup and delivery. Let us now call that original price matrix P_0 :

$$P_0 = \begin{pmatrix} 3 & 5 \\ 8 & 10 \\ 4 & 6 \end{pmatrix}_{3 \times 2}.$$

Now suppose there is another store location. That store has its own 3×2 price matrix. For example, its prices might be

$$P_1 = \begin{pmatrix} 4 & 6 \\ 7 & 9 \\ 5 & 7 \end{pmatrix}_{3 \times 2}.$$

We could perform our calculations with the two price matrices separately. For example, we could calculate the customer costs at the original store using OP_0 , or calculate the customer costs at the second store using OP_1 :

$$OP_0 = \begin{pmatrix} 26 & 38 \\ 43 & 66 \end{pmatrix}, \quad OP_1 = \begin{pmatrix} 30 & 42 \\ 42 & 56 \end{pmatrix}.$$

However, it is more convenient to combine the price matrices into a single tensor with an extra dimension. We imagine stacking the matrices P_0 and P_1 into a three-dimensional grid. The resulting tensor P has shape $2 \times 3 \times 2$: two stores, three resources, and two fulfillment methods.

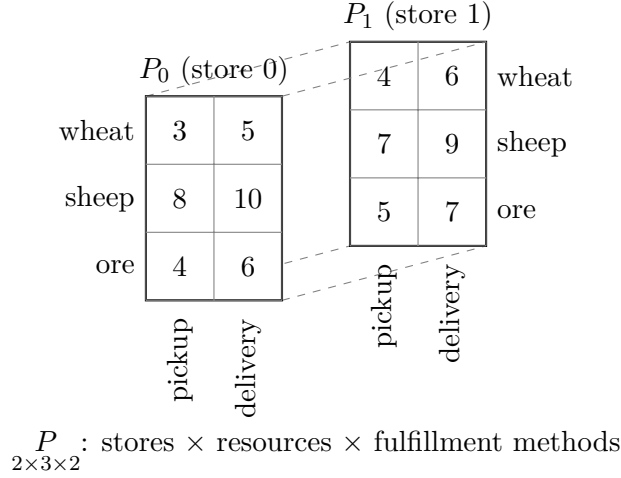


Figure 1: The price tensor P as a stack of two 3×2 matrices.

Figure 1 demonstrates this diagrammatically. In formal mathematical notation, we use Python-like notation to represent the various slices of the tensor. The tensor $P_{2 \times 3 \times 2}$ can be defined by giving its two 3×2 slices:

$$P_{0, :, :} = \begin{pmatrix} 3 & 5 \\ 8 & 10 \\ 4 & 6 \end{pmatrix},$$

$$P_{1, :, :} = \begin{pmatrix} 4 & 6 \\ 7 & 9 \\ 5 & 7 \end{pmatrix}.$$

Here, the first index selects the store, while the two colons select every resource and every fulfillment method for that store.

Let us make this example more interesting by adding two more competing stores. This gives us two more price matrices, and our new price tensor has shape $4 \times 3 \times 2$. Suppose the precise values are

$$P_{0, :, :} = \begin{pmatrix} 3 & 5 \\ 8 & 10 \\ 4 & 6 \end{pmatrix}, \quad P_{1, :, :} = \begin{pmatrix} 4 & 6 \\ 7 & 9 \\ 5 & 7 \end{pmatrix},$$

$$P_{2, :, :} = \begin{pmatrix} 2 & 4 \\ 9 & 11 \\ 3 & 5 \end{pmatrix}, \quad P_{3, :, :} = \begin{pmatrix} 5 & 7 \\ 6 & 8 \\ 7 & 9 \end{pmatrix}.$$

Together, these four slices define the tensor $P_{4 \times 3 \times 2}$.

Recall that we had the following order matrix for two customers:

$$O_{2 \times 3} = \begin{pmatrix} 2 & 1 & 3 \\ 1 & 4 & 2 \end{pmatrix}.$$

It specifies how much of each of the three resources is ordered by each of the two customers.

Through the magic of broadcasting, we can multiply this order matrix by the three-dimensional price tensor. Provided that our tensors are organized with the correct shapes, the result contains the costs for each customer at each store. The calculation has the shape

$$\begin{matrix} O & P \\ 2 \times 3 & 4 \times 3 \times 2 \end{matrix} \longrightarrow \begin{matrix} C \\ 4 \times 2 \times 2 \end{matrix},$$

where broadcasting applies O to each store slice:

$$C_{s,:} = OP_{s,:}, \quad s \in \{0, 1, 2, 3\}.$$

Explicitly, the four slices of the result are

$$\begin{aligned} C_{0,:} &= OP_{0,:} = \begin{pmatrix} 26 & 38 \\ 43 & 66 \end{pmatrix}, & C_{1,:} &= OP_{1,:} = \begin{pmatrix} 30 & 42 \\ 42 & 56 \end{pmatrix}, \\ C_{2,:} &= OP_{2,:} = \begin{pmatrix} 22 & 34 \\ 44 & 58 \end{pmatrix}, & C_{3,:} &= OP_{3,:} = \begin{pmatrix} 37 & 49 \\ 43 & 57 \end{pmatrix}. \end{aligned}$$

Thus, $C_{s,i,j}$ is the total cost for customer i at store s , using fulfillment method j .

The general rules for broadcasting when tensors are multiplied can be confusing. The particular pattern studied here is called *batch matrix multiplication*. It is useful to memorize this specific pattern because it is frequently used in neural networks.

In batch matrix multiplication, we have a matrix A that we would like to multiply by a large number of example matrices

$$\begin{matrix} B_0, B_1, \dots, B_{b-1} \\ n \times p \quad n \times p \quad n \times p \end{matrix}.$$

Here, b is the number of matrices in the batch. We stack the B_i matrices into a single three-dimensional tensor B and perform the multiplication. The resulting shapes are

$$\begin{matrix} A & B \\ m \times n & b \times n \times p \end{matrix} \longrightarrow \begin{matrix} C \\ b \times m \times p \end{matrix},$$

where

$$C_{i,:} = AB_{i,:}, \quad i \in \{0, 1, \dots, b-1\}.$$

Thus, A is broadcast across the batch dimension, and each slice of C is the ordinary matrix product of A with the corresponding slice of B .

In NumPy and PyTorch, it is extremely easy to perform batch matrix multiplication. We use the same matrix-multiplication operator, @ , as previously. Here is a complete NumPy example of the order-cost calculation for the four stores:

```
1 import numpy as np
2
3 O_numpy = np.array([
4     [2, 1, 3],
5     [1, 4, 2],
```

```

6  ])
7  P_numpy = np.array([
8      [[3, 5], [8, 10], [4, 6]],
9      [[4, 6], [7, 9], [5, 7]],
10     [[2, 4], [9, 11], [3, 5]],
11     [[5, 7], [6, 8], [7, 9]],
12 ])
13
14 C_numpy = O_numpy @ P_numpy
15 print(C_numpy)
16 # [[[26 38]
17  [43 66]]
18 #
19 # [[30 42]
20 # [42 56]]
21 #
22 # [[22 34]
23 # [44 58]]
24 #
25 # [[37 49]
26 # [43 57]]]
27 print(C_numpy.shape) # (4, 2, 2)

```

The corresponding PyTorch example is

```

1  import torch
2
3  O_torch = torch.tensor([
4      [2, 1, 3],
5      [1, 4, 2],
6  ])
7  P_torch = torch.tensor([
8      [[3, 5], [8, 10], [4, 6]],
9      [[4, 6], [7, 9], [5, 7]],
10     [[2, 4], [9, 11], [3, 5]],
11     [[5, 7], [6, 8], [7, 9]],
12 ])
13
14 C_torch = O_torch @ P_torch
15 print(C_torch)
16 # tensor([[[26, 38],
17           [43, 66]],
18 #
19 #          [[30, 42],
20 #          [42, 56]],
21 #
22 #          [[22, 34],
23 #          [44, 58]],

```

```

24 | #
25 | #         [[37, 49],
26 | #         [43, 57]]])
27 | print(C_torch.shape) # torch.Size([4, 2, 2])

```

Finally, let us examine a more realistic application of batch matrix multiplication. We imagine that the inputs to a neural network consist of two-dimensional matrices. In a transformer network, for example, each input string can be represented by a matrix whose rows represent its tokens. The different matrices represent different input strings that we would like to process in parallel.

The tokens are embedded into a different space by multiplication with a weight matrix W . This calls for batch matrix multiplication: we form the batch of input matrices into a three-dimensional tensor and then compute the embeddings. If b is the batch size, L is the number of tokens in each string, $|\mathcal{V}|$ is the number of possible token IDs, and d is the embedding dimension, then the shapes are

$$\underset{b \times L \times |\mathcal{V}|}{X} \underset{|\mathcal{V}| \times d}{W} \longrightarrow \underset{b \times L \times d}{E}, \quad E = XW.$$

Here, each slice $X_{i,:,:}$ is the token-representation matrix for one input string, and the same embedding matrix W is applied to every slice in the batch.